Guia do Turista Inteligente v2.0

Maria Clara, Roger Pierre, Antonio Carlos e Micael Cardoso

2026

CONTENTS

Guia do Turista Inteligente v2.0 3

1. Visão Geral da Evolução Arquitetural 3

2. Divisão Oficial da Equipe e Responsabilidades 5

3. Aluno 1 — Engenharia de Front-End & UX SaaS 5

3.1. Interface SaaS Split-Screen com HUD sobre o Mapa 5

3.2. Fórmula Matemática de Haversine no Cliente (geolocation.js) 6

3.3. Sessão Híbrida Não-Destrutiva (useAuth.js & visitanteStorage.js) 6

4. Aluno 2 — Backend Core ASGI, Concorrência & Resiliência 6

4.1. Duplo asyncio.gather() e Redução de Latência (< 800 ms) 6

4.2. Lifespan Context e Pool Persistente HTTPX (main.py) 7

4.3. Cancelamento Ativo (HTTP 499) e Circuit Breakers (circuit_breaker.py) 7

5. Aluno 3 — Contratos Pydantic v2, Altimetria & Custos 7

5.1. Contratos Estritos Pydantic v2 (schemas/roteiros.py) 7

5.2. Perfil Altimétrico Open-Elevation e Penalização de Consumo (altimetria_service.py) 8

5.3. Paradas Estratégicas (Margem 18%) e Rateio (postos_service.py & custos_service.py) 8

6. Aluno 4 — Streaming SSE, GPX 1.1, Persistência & QA/DevOps 8

6.1. Streaming em Tempo Real via Server-Sent Events (gemini_service.py) 8

6.2. Exportação XML GPX 1.1 (gpx_service.py) e Escrita Atômica (storage_service.py) 8

6.3. Observabilidade Prometheus e Suíte de 27 Testes Automatizados 9

7. Guia Rápido de Demonstração ao Vivo (Checklist da Apresentação) 9

CONFIDENTIAL

Guia do Turista Inteligente v2.0

Arquitetura Microsserviço FastAPI ASGI & SPA React 19 — Dossiê Técnico e Roteiro de Defesa

Disciplina: Automação Web em Python (Unidade I & Unidade II)
Docente: Prof. Maykol Sampaio
Instituição: Universidade Federal do Piauí (UFPI) — Curso de Tecnologia em Análise e Desenvolvimento de Sistemas (TADS)
Repositório Oficial: `https://github.com/MICAELCARDOSOREIS/guia-turista-inteligentev2`

---

1. Visão Geral da Evolução Arquitetural

O projeto **Guia do Turista Inteligente** passou por uma refatoração estrutural completa entre a **Unidade I** e a **Unidade II**, evoluindo de um monólito síncrono para uma arquitetura moderna orientada a microsserviços assíncronos e cliente rico desacoplado:

CONFIDENTIAL

Dimensão Técnica	Unidade I (Monólito Flask WSGI)	Unidade II (FastAPI ASGI + React 19 SPA)
Protocolo de Servidor	WSGI Síncrono Bloqueante (`Flask`)	ASGI Assíncrono Não-Bloqueante (`FastAPI` + `Uvicorn`)
Orquestração de APIs	Sequencial (`httpx.Client` bloqueando a thread)	Paralela com duplo `asyncio.gather()` e `httpx.AsyncClient`
Latência Média (`/planejar`)	~3.200 ms a 4.500 ms	**< 800 ms** (cabeçalho auditável `X-Process-Time`)
Entrega do Guia com IA	Espera total antes de renderizar o HTML	**Streaming em Tempo Real via SSE** (`text/event-stream`)
Interface de Usuário	Templates Jinja2 SSR com recarregamento (PRG)	**SPA React 19 + Vite** com Mapa Leaflet + HUD Flutuante
Validação de Contratos	`request.form.get()` manual com `try/except`	**Pydantic v2** (Rust `pydantic-core`) com `@field_validator`
Resiliência e Proteção	`try/except` e `threading.Lock()` local	**Circuit Breakers** (`CLOSED` / `OPEN` / `HALF_OPEN`) + HTTP 409 Lock
Cálculo Energético	Não contemplado	**Altimetria Open-Elevation**, margem de 18% e rateio de custos
Observabilidade e QA	Sem suíte automatizada	Probes `/healthz`, `/readyz`, `/metrics` Prometheus + **27 testes**

CONFIDENTIAL

2. Divisão Oficial da Equipe e Responsabilidades

Papel	Integrante	Matrícula	Módulos sob Responsabilidade
Aluno 1 — Frontend Developer	**Maria Clara Almeida Martins**	`2025116TADS0012`	SPA React 19 (`frontend/src/`), Mapa Interativo Leaflet com HUD (`MapaRota.jsx`), Geolocalização Geodésica Haversine (`geolocation.js`), Sessão Híbrida Não-Destrutiva (`useAuth.js`) e Design System SaaS (`index.css`).
Aluno 2 — Backend Core Engineer	**Roger Pierre Reis Silva**	`2025116TADS0010`	Entrypoint ASGI e Lifespan Pool (`main.py`), Orquestração concorrente `/planejar` (`roteiros.py`), Circuit Breakers (`circuit_breaker.py`), `clima_service.py` e `rota_service.py` .
Aluno 3 — Data & Altimetry Engineer	**Antonio Carlos Gomes**	`2025116TADS0024`	Schemas Pydantic v2 (`schemas/roteiros.py` , `schemas/auth.py`), Integração Open-Elevation e penalização altimétrica (`altimetria_service.py`), paradas por autonomia com reserva de 18% (`postos_service.py`) e rateio (`custos_service.py`).
Aluno 4 — DevOps, QA & Streaming Lead	**Micael Cardoso Reis**	`2025116TADS0041`	Streaming SSE com Gemini (`gemini_service.py`), Exportação XML GPX 1.1 (`gpx_service.py`), Persistência atômica JSON (`storage_service.py`), Métricas Prometheus (`/metrics`) e Suítes Pytest (21 testes) + Vitest (6 testes).

---

3. Aluno 1 — Engenharia de Front-End & UX SaaS

Integrante: Maria Clara Almeida Martins (`2025116TADS0012`)

3.1. INTERFACE SAAS SPLIT-SCREEN COM HUD SOBRE O MAPA

A SPA em React 19 (`:5173`) adota o padrão de produtos modernos de planejamento de viagens (como *Wanderlog*, *Airbnb Trips* e *Apple Maps*):

CONFIDENTIAL

- **Coluna Lateral Fixa (`planner-sidebar`):** Reúne o formulário unificado de Origem/Destino com botão de inversão rápida (`⇅ Inverter`), seletor em pílula (`Combustão` vs `100% Elétrico EV`), cálculo instantâneo da autonomia segura (descontando 18% de reserva) e histórico de viagens recentes.
- **Palco Principal (`main-stage`):** Mantém o mapa interativo Leaflet sempre visível, acompanhado de uma barra flutuante (**HUD**) para troca de rotas alternativas e exportação rápida (Google Maps, WhatsApp e GPX 1.1), faixa de 4 KPIs executivos, painel de perfil altimétrico e linha do tempo de atividades por turno (`07h-12h`, `13h-18h`, `19h-23h`).

3.2. FÓRMULA MATEMÁTICA DE HAVERSINE NO CLIENTE (`GEOLOCATION.JS`)

Para sugerir destinos próximos sem onerar o backend com chamadas de rede desnecessárias, o front-end calcula a distância geodésica esférica usando o raio médio da Terra ($R = 6.371\text{ km}$):

- Filtra e ordena os destinos mais próximos das coordenadas obtidas via `navigator.geolocation`.
- Sinaliza automaticamente trechos com distância linear superior a **400 km**, alertando sobre paradas obrigatórias de descanso e abastecimento/recarga.

3.3. SESSÃO HÍBRIDA NÃO-DESTRUTIVA (`USEAUTH.JS` & `VISITANTESTORAGE.JS`)

Quando um usuário autenticado via Google OAuth decide testar a plataforma em **Modo Visitante Temporário** (com TTL regressivo de 2 horas), sua credencial Google não é destruída: ela permanece preservada em `guia_turista_saved_google_user`, permitindo restaurar o perfil principal em 1 clique.

---

4. Aluno 2 — Backend Core ASGI, Concorrência & Resiliência

Integrante: Roger Pierre Reis Silva (`2025116TADS0010`)

4.1. DUPLO `ASYNCIO.GATHER()` E REDUÇÃO DE LATÊNCIA (< 800 MS)

No endpoint `POST /api/v1/roteiros/planejar` (`backend/app/routers/roteiros.py`), eliminamos o gargalo sequencial dividindo as chamadas I/O em dois estágios paralelos sobre o Event Loop ASGI:

CONFIDENTIAL

1. **Estágio 1 (Geocodificação Paralela):** Dispara simultaneamente `buscar_coordenadas_async(origem)` e `buscar_coordenadas_async(destino)`.
2. **Estágio 2 (Clima + Malha Viária Paralelos):** Com as coordenadas em mãos, executa simultaneamente `obter_clima_async` (Open-Meteo Forecast) e `calcular_rota_osrm_async` (OSRM Driving API).

4.2. LIFESPAN CONTEXT E POOL PERSISTENTE HTTPX (`MAIN.PY`)

Em vez de abrir um socket TCP/TLS a cada requisição, o gerenciador `@asynccontextmanager async def lifespan(app: FastAPI)` inicializa uma única instância de `httpx.AsyncClient` compartilhada em `app.state.http_client` com `Limits(max_keepalive_connections=50, max_connections=100)`, injetada nos endpoints via `Depends(obter_http_client)`.

4.3. CANCELAMENTO ATIVO (`HTTP 499`) E CIRCUIT BREAKERS (`CIRCUIT_BREAKER.PY`)

- **Desconexão Prematura:** O pipeline verifica `if await request.is_disconnected(): raise HTTPException(status_code=499)`, abortando imediatamente o processamento se o cliente fechar a aba.
- **Disjuntor de Circuito:** Protege as chamadas externas com máquina de estados (`CLOSED` → `OPEN` após 3 falhas consecutivas → `HALF_OPEN` após janela de recuperação), acionando contingência local em menos de 1 ms caso um provedor externo caia.

---

5. Aluno 3 — Contratos Pydantic v2, Altimetria & Custos

Integrante: Antonio Carlos Gomes (`2025116TADS0024`)

5.1. CONTRATOS ESTRITOS PYDANTIC V2 (`SCHEMAS/ROTEIROS.PY`)

Todos os payloads de entrada e saída são validados pelo motor em Rust do Pydantic v2:

- O decorador `@field_validator("origem_uf", "destino_uf")` converte siglas para maiúsculas, aceita nomes completos de estados e rejeita imediatamente qualquer UF inexistente no Brasil (ex.: `"XX"`) com **HTTP 422 Unprocessable Entity**.
- Limites numéricos estritos (`Field(gt=0, le=50)` para consumo, `Field(ge=1, le=8)` para passageiros) impedem divisões por zero ou payloads malformados.

CONFIDENTIAL

5.2. PERFIL ALTIMÉTRICO OPEN-ELEVATION E PENALIZAÇÃO DE CONSUMO (`ALTIMETRIA_SERVICE.PY`)

A partir da polilinha da rota, o serviço amostra pontos ao longo da rodovia e consulta a API **Open-Elevation**:

- Calcula **ganho acumulado de subida (m)**, **perda de descida (m)**, **altitude máxima (m)** e **desnível líquido (m)**.
- A função `calcular_penalizacao_altimetrica()` calcula a razão de aclive (m/km) e aplica uma **penalização energética de até +25%** no consumo do veículo em trechos serranos.

5.3. PARADAS ESTRATÉGICAS (MARGEM 18%) E RATEIO (`POSTOS_SERVICE.PY` & `CUSTOS_SERVICE.PY`)

- **Autonomia Segura:** Dimensiona paradas de abastecimento ou eletropostos sempre que o alcance atinge **82% da capacidade total** (preservando 18% de margem de segurança).
- **Orçamento e Rateio:** Combina o consumo penalizado pelo relevo com a estimativa regional de pedágios por UF e divide o valor total entre **1 a 8 ocupantes**.

---

6. Aluno 4 — Streaming SSE, GPX 1.1, Persistência & QA/DevOps

Integrante: Micael Cardoso Reis (`2025116TADS0041`)

6.1. STREAMING EM TEMPO REAL VIA SERVER-SENT EVENTS (`GEMINI_SERVICE.PY`)

O endpoint `GET /api/v1/roteiros/stream-guia` utiliza `StreamingResponse(..., media_type="text/event-stream")` para transmitir o roteiro cultural gerado pelo Google Gemini (ou pelo motor determinístico de contingência) bloco a bloco (`yield f"data: {json.dumps({'chunk': bloco})}\n\n"`), entregando o primeiro parágrafo na tela em menos de 300 ms.

6.2. EXPORTAÇÃO XML GPX 1.1 (`GPX_SERVICE.PY`) E ESCRITA ATÔMICA (`STORAGE_SERVICE.PY`)

- **GPX 1.1 Oficial:** O endpoint `/api/v1/roteiros/exportar/gpx` gera um arquivo XML compatível com o schema `http://www.topografix.com/GPX/1/1`, contendo `<wpt>` de origem/destino e `<trkseg>` com `<trkpt>` para uso em navegadores Garmin, Strava ou OsmAnd.

- **Persistência Thread-Safe & Atômica:** O `storage_service.py` combina `threading.Lock()` com escrita em arquivo temporário (`.tmp`) seguida de substituição atômica POSIX (`caminho_tmp.replace(caminho)`), prevenindo qualquer corrupção do `viagens.json` .
- **Bloqueio de Duplo Clique (`HTTP 409 Conflict`):** Um `asyncio.Lock()` sobre o conjunto `REQUISICOES_ATIVAS` bloqueia submissões duplicadas simultâneas do mesmo usuário.

6.3. OBSERVABILIDADE PROMETHEUS E SUÍTE DE 27 TESTES AUTOMATIZADOS

- **Endpoints de Telemetria (`telemetria.py`):** `/healthz` (liveness), `/readyz` (readiness dos circuit breakers) e `/metrics` (formato texto Prometheus).
- **100% de Aprovação nas Suítes Automatizadas:**
  - **Backend (`pytest`):** `21 passed` (`test_roteiros.py` , `test_schemas.py` , `test_services.py` , `test_settings.py` , `test_streaming_gpx.py`).
  - **Frontend (`vitest`):** `6 passed` (`config.test.js` , `geolocation.test.js` , `RoteiroForm.test.jsx`).

---

7. Guia Rápido de Demonstração ao Vivo (Checklist da Apresentação)

1. **Mostrar a Interface SaaS (`http://localhost:5173`):**
   - Destacar o mapa sempre visível, clicar em **Destinos próximos** (Haversine), inverter Origem/Destino (`⇅ Inverter`), alternar entre **Combustão** e **100% Elétrico (EV)** e clicar em **Planejar Viagem**.
2. **Destacar os Resultados em Tempo Real:**
   - Apontar para os 4 KPIs no topo, os marcadores de paradas no mapa, o gráfico de **Perfil Altimétrico**, a simulação de **Viajantes (1x a 8x)** no rateio de custos, a programação por turnos (`07h–12h` , `13h–18h` , `19h–23h`) e o **Check-in** nos pontos turísticos.
3. **Demonstrar a Sessão Híbrida e Exportação GPX:**
   - Alternar para **Modo Visitante** (mostrando o cronômetro de 2h), clicar em **Retomar Maria** e baixar o arquivo **GPX 1.1** na barra flutuante do mapa.
4. **Demonstrar o Backend e a Suíte de Testes no Terminal:**
   - Mostrar o Swagger UI em `http://localhost:8001/docs` e `/metrics` .
   - Executar `PYTHONPATH=. .venv/bin/pytest tests/ -v` (21 testes verdes) e `npm test` (6 testes verdes).

CONFIDENTIAL